

Developing A Self-Explanatory Transformer

Rasha Karakchi

Dept. of Computer Science and Engineering
University of South Carolina
Columbia, USA
karakchi@cec.sc.edu

Ryan Karbowniczak

Dept. of Computer Science and Engineering
University of South Carolina
Columbia, USA
karbownr@email.sc.edu

Abstract—While IoT devices provide significant benefits, their rapid growth results in larger data volumes, increased complexity, and higher security risks. To manage these issues, techniques like encryption, compression, and mapping are used to process data efficiently and securely. General-purpose and AI platforms handle these tasks well, but mapping in natural language processing is often slowed by training times. This work explores a self-explanatory, training-free mapping transformer based on non-deterministic finite automata, designed for Field-Programmable Gate Arrays (FPGAs). Besides highlighting the advantages of this proposed approach in providing real-time, cost-effective processing and dataset-loading, we also address the challenges and considerations for enhancing the design in future iterations.

Index Terms—Transformer, FPGA, non-deterministic automata

I. INTRODUCTION

The rapid increase in IoT devices, including sensors, cameras, and smart appliances, is leading to a significant surge in data generation [1]. Each device continuously produces data streams, resulting in an exponential rise in overall data volume. Edge computing addresses this challenge by processing data closer to its source, which helps reduce latency and bandwidth usage. However, this also requires large volumes of data to be processed, filtered, and analyzed in real-time, adding to the system's complexity. As data generation grows, so do the challenges of securing it. Edge computing introduces new points of vulnerability, necessitating robust security measures to protect data at the edge, during transmission, and within central data centers. This increase in data has made data transformation a crucial aspect of modern computing. Techniques like data compression, filtration, encryption, and mapping are essential for ensuring faster, more reliable, and efficient data movement across systems. Data mapping, a key type of data transformation, is particularly important in natural language processing, where it involves converting or aligning textual data from one format or representation to another. This process is vital for tasks such as language translation, converting text into structured data, or aligning datasets from various sources.

Automata-based transformation, a rule-based and training-free model, offers an efficient approach to data transformation and mapping by processing streamed input and generating equivalent output based on preloaded rules. General-purpose platforms have shown good results with deterministic

automata-based transformations, where each input sequence produces only one output sequence [2], [3]. However, some applications involve non-deterministic outputs like traveling salesman problem where multiple paths are explored, which are challenging to execute on general-purpose computing platforms like CPUs and GPUs due to the unpredictable memory accesses. Consequently, there is a shift towards domain-specific platforms like FPGAs and ASICs, which offer customization, on-chip memory usage, and parallel processing capabilities [4].

This work presents a preliminary work on developing a data transformer using non-deterministic finite automata transduction techniques. Automata graph consists of states and edges carrying a combination of input symbol and output symbol. We design our accelerator as a two-dimensional array of processor elements (PEs) where each processor represents an edge, adapted from Micron's AP [5], [6]. Switch-based interconnection is designed to allow processor elements to communicate with and through neighbor PEs. We tested our design on programmable logic of Zync 104 FPGA and we measured the hardware resources and power consumed for the proposed approach.

II. BACKGROUND

The Finite State Transducer (FST) is a finite state machine equipped with two memory tapes, similar to the input and output tapes of Turing machines. The processing unit acts as the state machine, moving between states based on input symbols and generating corresponding outputs.

A Finite State Transducer (FST) is a 7-tuple $(Q, \Sigma, \delta, \omega, \Gamma, q_0, F)$ where Q set of states, Σ set of symbols, $\delta: Q \times \Sigma \rightarrow P(Q)$ the transition function, $\omega: Q \times \Sigma \rightarrow \Gamma$ the output function (transduction), $q_0 \in Q$ the start state, and $F \subseteq Q$ a set of accepting states.

In FST, data transformation is guided by an encoded relation, known as a rational or regular relation, which maps a set of input symbols to a set of output symbols. During each clock cycle, the automaton transitions based on (1) the current state and (2) whether the input symbol matches the edge label. When a match occurs and the automaton reaches a "report" state, it outputs a sequence.

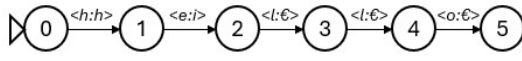


Fig. 1. Example of Finite State Transducer

Figure 1 illustrates an example of an FST with six states (where state 0 is the start state and state 5 is the report state) that recognizes the pattern "hello" and transforms it into "hi". Each transition is labeled with a *Symbol1:Symbol2* pair, where *Symbol1* is the input symbol triggering the transition, and *Symbol2* is the output symbol generated. In this FST, each transition verifies the input symbol and, if it matches, produces the corresponding output symbol.

III. PROPOSED APPROACH

Our design draws inspiration from NAPOLY [7]–[9], which is a non-deterministic finite automata overlay. It consists of an array of processor elements (PEs), each containing a 256×1 – bit distributed on-chip RAM that stores the input symbol for the edge. Additionally, each PE is equipped with programmable switches that enable communication with neighboring PEs. The output function of the transformer is implemented using a separate on-chip distributed RAM.

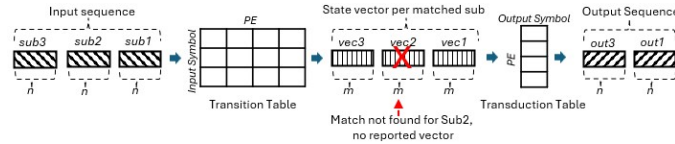


Fig. 2. Transformer Framework

Given the non-deterministic nature of the system, the path (i.e., the number of active states at one time) must be tracked while the input sequence is streamed to determine the corresponding output sequence. To manage this, we split the input sequence into fixed-length sub-sequences of size n , as depicted in Figure 2. For each sub-sequence, a vector is created to log the activated PE IDs. The maximum size of this vector is $O(m^2)$, where m represents the total number of PEs. If a match is detected within the array, the vector is flushed from the engine to the FIFO, and then to the transduction RAM, to generate the output sequence. After the vector is flushed, a new sub-sequence begins streaming into the array. If no match is found (i.e., no sequence is detected), the vector is discarded, and a new input sub-sequence begins streaming, as illustrated in Figure 2 for *Sub2*.

$$\begin{aligned}
 \text{Execution time} = & \text{Time_to_flush_sub_sequence} (1_clk \times n) \\
 & + \text{transition_time} (2_clk \times n) \\
 & + \text{time_to_flush_out_vector} (1_clk) \\
 & + \text{transduction_time} (1_clk \times m) \\
 & + \text{time_to_flush_output} (1_clk \times n)
 \end{aligned} \quad (1)$$

In this design, we assume that each input symbol corresponds to exactly one output symbol. Consequently, the

lengths of the output sub-sequences (*out1*, *out3*) are equal to n . This design is training-free and straightforward because new datasets are uploaded during reconfiguration, leveraging the on-chip memory to store the transition and output symbols.

The execution time of the design is determined by the total time required to generate the output for a single sub-sequence. The total execution time is calculated as illustrated in Eq. (1).

IV. RESULTS AND CONCLUSION

We implemented the design on the programmable logic of the Zynq 104 FPGA. Figure 3 illustrates the hardware resources used, power consumption, and memory utilization across various array sizes. The figure also demonstrates the increase in hardware resources (LUTs and Registers) relative to the array size, while power consumption remains at 79% for all configurations since all arrays were run at the same maximum frequency of 500 MHz.

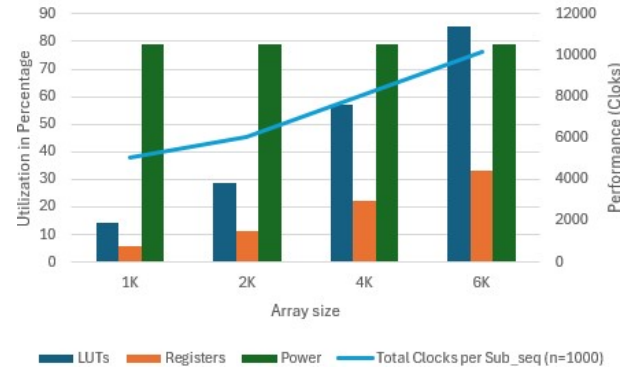


Fig. 3. Preliminary Hardware Results

There are several limitations in the current design that we plan to address in future work. First, the PEs can only communicate with their immediate neighbors. While this reduces the number of required input/output wires and allows for larger array sizes, it also prevents direct communication between non-adjacent PEs, increasing the likelihood of congestion. Another limitation is the size and number of vectors generated for each sub-sequence to record the sequence track, which is constrained. In applications with many active states at once (i.e., multiple potential paths), tracking all paths becomes challenging which we aim to enhance for our future work. We also plan to explore streaming multiple input batches to improve performance.

ACKNOWLEDGMENT

Thanks to the Center for Integrative and Experiential Learning at the University of South Carolina for supporting this work through the Carolina Engage Grant.

REFERENCES

- [1] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief. A survey on mobile edge computing: The communication perspective. *IEEE Commun. Surv. Tutor.*, 19(4):2322–2358, 2017.

- [2] T. Nguyen and M. Becchi. A gpu-accelerated data transformation framework rooted in pushdown transducers. In *Proc. IEEE Int. Conf. High Performance Comput., Data, Anal. (HiPC)*, pages 215–225. IEEE, 2022.
- [3] M. Nourian, T. Nguyen, A. A. Chien, and M. Becchi. Data transformation acceleration using deterministic finite-state transducers. In *Proc. IEEE Int. Conf. Big Data*, pages 141–150. IEEE, 2022.
- [4] R. Karakchi. A scratchpad spiking neural network accelerator. In *2024 IEEE 3rd International Conference on Computing and Machine Intelligence (ICMI)*, pages 1–5, 2024.
- [5] K. Wang, K. Angstadt, C. Bo, N. Brunelle, E. Sadredini, T. Tracy, J. Wadden, M. Stan, and K. Skadron. An overview of micron’s automata processor. In *Proc. IEEE/ACM/IFIP Int. Conf. Hardware/Software Code-sign Syst. Synth. (CODES+ISSS)*, pages 1–3, 2016.
- [6] R. Karakchi, L. O. Richards, and J. D. Bakos. A dynamically reconfigurable automata processor overlay. In *Proc. Int. Conf. Reconfigurable Comput. FPGAs (ReConFig)*, pages 1–8. IEEE, 2017.
- [7] R. Karakchi and J. D. Bakos. Napoly: A non-deterministic automata processor overlay. In *ACM Trans. Reconfigurable Technol. Syst.*, volume 16, pages 1–25. ACM, 2023.
- [8] R. Karbowiczak and R. Karakchi. A scored non-deterministic finite automata processor for sequence alignment. *arXiv preprint*, 2024. eprint 2410.19758.
- [9] R. Karakchi, C. Daniels, and J. Bakos. An overlay architecture for pattern matching. In *Proc. IEEE Int. Conf. Application-Specific Syst., Archit. Processors (ASAP)*, volume 2160, pages 165–172. IEEE, 2019.